
Modül 05 — Spring JDBC ve PostgreSQL

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Spring’de Veritabanı Erişim Katmanları	2
2.2 Transaction’lar	2
2.3 Geliştirme ve Test Veritabanı	3
3. Adım Adım Örnekler	3
3.1 Docker Compose ile Veritabanı	3
3.2 Flyway ile Şema Yönetimi	4
3.3 JdbcClient ile SQL	5
3.4 Spring Data JDBC ile Aggregate	6
3.5 Transaction’lar	7
3.6 @TransactionalEventListener	8
3.7 Testcontainers ile Test	9
4. Sık Yapılan Hatalar ve En İyi Pratikler	10
5. Özet	11
6. İleri Okuma	11

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Geliştirme veritabanını Spring Boot'un Docker Compose desteğiyle otomatik başlatmak
- Veritabanı şemasını Flyway migration'larıyla sürümlemek
- JdbcClient ile SQL sorguları yazmak ve sonuçları record'lara eşlemek
- Spring Data JDBC ile bir aggregate'i tek parça olarak kaydetmek ve okumak
- @Transactional ile tutarlı işlemler yazmak, rollback ve propagation davranışını bilmek
- @TransactionalEventListener ile yalnızca commit edilen işlere tepki vermek
- Gerçek bir PostgreSQL ile Testcontainers kullanarak test yazmak

Ön koşullar: Modül 01-03, temel SQL · **Tahmini süre:** 5 saat · **Docker gerekir**

2. Kavramlar

2.1 Spring'de Veritabanı Erişim Katmanları

Katman	Siz ne yazarsınız?	Ne zaman?
JdbcClient	SQL	SQL üzerinde tam kontrol, raporlar, toplu işlemler
Spring Data JDBC	Aggregate sınıfları + repository arayüzü	Basit, öngörülebilir nesne-tablo eşleme
Spring Data JPA (Modül 06)	Entity'ler + repository	Karmaşık ilişkiler, lazy loading, önbellek

Hepsinin altında bir DataSource (varsayılan olarak **HikariCP** bağlantı havuzu) ve JDBC sürücüsü vardır. Spring, SQL hatalarını veritabanından bağımsız bir exception hiyerarşisine çevirir (ör. DataIntegrityViolationException).

2.2 Transaction'lar

Bir transaction'daki değişikliklerin ya **hepsi** kalıcı olur (commit) ya da **hiçbiri** olmaz (rollback). @Transactional bunu bir proxy üzerinden yönetir:

- Metot başlarken transaction açılır, normal biterse commit edilir.
- RuntimeException (ve Error) fırlarsa rollback yapılır. **Checked exception'larda varsayılan olarak commit edilir.**

Propagation	Anlamı
REQUIRED (varsayılan)	Varsa mevcut transaction'a katıl, yoksa yeni aç
REQUIRES_NEW	Mevcut transaction'ı askıya al, kendi transaction'ını aç ve kapat
MANDATORY	Mevcut bir transaction şart, yoksa hata
NOT_SUPPORTED	Transaction dışında çalış

2.3 Geliştirme ve Test Veritabanı

- **Geliştirme:** spring-boot-docker-compose bağımlılığı, spring-boot:run sırasında kök compose.yaml'daki PostgreSQL'i başlatır. Bağlantı bilgileri container'dan okunur, spring.datasource.* yazmanız gerekmez.
- **Test:** Testcontainers her test çalıştırmasında tek kullanımlık, gerçek bir PostgreSQL başlatır. @ServiceConnection, bağlantı bilgilerini Boot'a verir. H2 gibi bir taklit veritabanı yerine gerçek veritabanıyla test etmek, veritabanına özel davranışları (kısıtlar, tipler, hata kodları) da yakalar.

3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın:

```
./mvnw -pl modules/05-data-jdbc-postgres/lesson -am spring-boot:run
```

3.1 Docker Compose ile Veritabanı

Amaç: Tek komutla, elle container başlatmadan bir PostgreSQL ile çalışmak.

```
docker:
  compose:
    # Reuse the root compose.yaml; only the listed profiles are started.
    file: ../../../../compose.yaml
    profiles:
      active: postgres
    # Keep containers running between restarts (stop them with: docker compose --profile
    ↪ all down)
    lifecycle-management: start-only
```

Beklenen çıktı:

```
... DockerCli : Container spring-boot-edu-postgres-1 Started
... DockerCli : Container spring-boot-edu-postgres-1 Healthy
== 3.1 DataSource (from Docker Compose)
jdbc:postgresql://127.0.0.1:5432/bookstore?ApplicationName=05-data-jdbc-postgres ·
↪ PostgreSQL 18.6 ... · pool HikariDataSource
```

i Not

Kök compose.yaml dosyasında PostgreSQL servisi pgvector/pgvector imajını kullanır. Boot servisi imaj adından tanır ve bu imaj “postgres” adını taşımaz. Bu yüzden servise org.springframework.boot.service-connection: postgres etiketi eklenmiştir.

3.2 Flyway ile Şema Yönetimi

Amaç: Şemayı kodla birlikte sürümlemek ve her ortamda aynı sırayla uygulamak.

src/main/resources/db/migration/ altındaki V<sürüm>__<açıklama>.sql dosyaları, uygulama başlarken sırayla çalıştırılır. Uygulananlar flyway_schema_history tablosuna yazılır:

```
create table author (
    id    bigserial primary key,
    name text      not null
);

create table book (
    id          bigserial primary key,
    isbn        varchar(13) not null unique,
    title       text       not null,
    author_id   bigint      references author (id),
    price       numeric(10,2) not null check (price >= 0),
    stock       integer     not null default 0 check (stock >= 0) -- the database guards
↪ the rule too
);
```

Beklenen çıktı:

```
... DbMigrate : Migrating schema "public" to version "1 - create schema"
... DbMigrate : Migrating schema "public" to version "2 - seed books"
== 3.2 Flyway
V1 create schema
V2 seed books
```

Testi: FlywayMigrationTest

!! Dikkat

Uygulanmış bir migration dosyasını **asla değiştirmeyin**. Flyway checksum farkını görür ve uygulamayı başlatmaz. Değişiklik için yeni bir sürüm (V3__...) ekleyin.

3.3 JdbcTemplate ile SQL

Amaç: SQLi kendiniz yazmak ama bağlantı, parametre ve eşleme işini Spring'e bırakmak.

```
public List<Book> findAll() {
    return jdbc.sql("select id, isbn, title, price, stock from book order by id")
        .query(Book.class) // columns → record components
        .list();
}

public Optional<Book> findByIsbn(String isbn) {
    return jdbc.sql("select id, isbn, title, price, stock from book where isbn = :isbn")
        .param("isbn", isbn) // named parameter: no SQL injection
        .query(Book.class)
        .optional();
}
```

Bir join sonucu, yalnızca o sorgu için tanımlanmış bir record'a eşlenebilir. author_name sütunu authorName bileşenine gider:

```
public List<BookWithAuthor> findWithAuthors() {
    return jdbc.sql("""
        select b.title, a.name as author_name
        from book b join author a on a.id = b.author_id
        order by b.title""")
        .query(BookWithAuthor.class)
        .list();
}
```

Veritabanının ürettiği anahtarı almak için KeyHolder kullanılır:

```
public long insert(Book book) {
    KeyHolder keys = new GeneratedKeyHolder();
    jdbc.sql("insert into book (isbn, title, price, stock) values (:isbn, :title, :price,
↪ :stock)")
        .paramSource(book) // parameters from the record's
        ↪ components
        .update(keys, "id"); // ask PostgreSQL for the
        ↪ generated id
}
```

```
    return keys.getKeyAs(Long.class);  
}
```

Beklenen çıktı:

```
== 3.3 JdbcClient  
Book[id=1, isbn=9780134685991, title=Effective Java, price=89.90, stock=5]  
...  
BookWithAuthor[title=Effective Java, authorName=Joshua Bloch]
```

Testi: book/JdbcBookRepositoryTest. Negatif stok, veritabanındaki check kısıtına takılır ve DataIntegrityViolationException olarak gelir.

3.4 Spring Data JDBC ile Aggregate

Amaç: Birlikte değişen nesneleri (sipariş ve satırları) tek bir birim olarak saklamak.

Aggregate kökü ve onun alt koleksiyonu:

```
@Table("purchase_order") // "order" is a reserved word  
↪ in SQL  
public record PurchaseOrder(  
    @Id @Nullable Long id, // null → INSERT, otherwise UPDATE  
    String customerEmail,  
    Instant createdAt,  
    @MappedCollection(idColumn = "purchase_order") Set<OrderLine> lines) {  
  
    public static PurchaseOrder create(String customerEmail, Set<OrderLine> lines) {  
        return new PurchaseOrder(null, customerEmail, Instant.now(), lines);  
    }  
  
    public PurchaseOrder withId(Long newId) { // Spring Data uses this to set  
        ↪ the generated id  
        return new PurchaseOrder(newId, customerEmail, createdAt, lines);  
    }  
}
```

Her aggregate kökü için bir repository yazılır. Uygulamayı Spring Data üretir:

```
public interface PurchaseOrderRepository extends ListCrudRepository<PurchaseOrder, Long>  
↪ {  
  
    List<PurchaseOrder> findByCustomerEmail(String customerEmail); // derived from  
    ↪ the method name  
}
```

```

@Query("""
    select po.id from purchase_order po
    join order_line ol on ol.purchase_order = po.id
    group by po.id
    having sum(ol.quantity * ol.unit_price) > :minimum""")
List<Long> findIdsWithTotalAbove(BigDecimal minimum);           // hand-written
    ↳ SQL when needed
}

```

save(order) hem purchase_order hem order_line satırlarını yazar. findById ikisini birlikte okur. Satırların kendi repository'si yoktur, çünkü aggregate'in parçasıdır.

Beklenen çıktı:

```

== 3.4 Spring Data JDBC aggregate
order #1 with 1 line(s), total 240.00

```

Testi: order/PurchaseOrderRepositoryTest (@DataJdbcTest)

★ İpucu

Spring Data JDBC, JPA'dan bilerek daha basittir: lazy loading, "dirty checking" ve birinci seviye önbellek yoktur. Ne kaydederseniz o yazılır. Bu da davranışı öngörülebilir kılar.

3.5 Transaction'lar

Amaç: Stok düşme ve sipariş kaydını tek bir tutarlı işlem yapmak.

```

@Transactional                                           // begin ... commit, or rollback on a
    ↳ RuntimeException
public long placeOrder(String customerEmail, Map<String, Integer> quantities) {
    audit.record("Checkout started by " + customerEmail); // separate transaction
    ↳ (REQUIRES_NEW)

    Set<OrderLine> lines = new HashSet<>();
    quantities.forEach((isbn, quantity) -> {
        Book book = books.findByIsbn(isbn).orElseThrow(() -> new
            ↳ IllegalArgumentException("Unknown ISBN " + isbn));
        if (book.stock() < quantity) {
            throw new OutOfStockException(isbn, quantity, book.stock()); // →
            ↳ everything is rolled back
        }
        books.changeStock(isbn, -quantity);
        lines.add(new OrderLine(isbn, quantity, book.price()));
    }
}

```

```
});

long orderId = Objects.requireNonNull(orders.save(PurchaseOrder.create(customerEmail,
    ↪ lines)).id());
events.publishEvent(new OrderPlacedEvent(orderId, customerEmail)); // delivered
↪ after commit
return orderId;
}
```

Denetim kaydı, sipariş başarısız olsa bile kalıcı olmalıdır. Bu yüzden kendi transaction'ında yazılır:

```
@Component
public class AuditLog {

    private final JdbcClient jdbc;

    public AuditLog(JdbcClient jdbc) {
        this.jdbc = jdbc;
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW) // suspend the caller's
    ↪ transaction, commit on its own
    public void record(String message) {
        jdbc.sql("insert into audit_log (message) values (?)").param(message).update();
    }
}
```

3.6 @TransactionalEventListener

Amaç: Bir işe yalnızca **commit edildikten sonra** tepki vermek. Geri alınmış bir sipariş için müşteriye e-posta gitmemelidir.

```
@Component
public class OrderNotifier {

    private final JdbcClient jdbc;

    public OrderNotifier(JdbcClient jdbc) {
        this.jdbc = jdbc;
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT) // the default
    ↪ phase
    @Transactional(propagation = Propagation.REQUIRES_NEW) // the original
    ↪ transaction is over
```

```

public void onOrderPlaced(OrderPlacedEvent event) {
    jdbc.sql("insert into notification (message) values (?)")
        .param("Siparişiniz alındı / Order received: #" + event.orderId() + " → "
            + event.customerEmail())
        .update();
}
}

```

Beklenen çıktı (3.5 ve 3.6, temiz bir veritabanıyla ilk çalıştırma):

```

== 3.5 + 3.6 Transactions and transactional events
checkout OK → order #2, stock of Spring in Action now 3
checkout FAILED → Only 2 of 99 copies of 9780134757599 in stock
audit_log rows: 2 (both attempts, thanks to REQUIRES_NEW)
notification rows: 1 (only after a commit)

```

Testi: checkout/CheckoutServiceTest. Başarılı siparişte her şeyin commit edildiğini doğrular. Başarısız siparişte stok ve siparişin geri alındığını, denetim kaydının ise kaldığını doğrular.

i Not

Veriler bir Docker volume'unda kalır ve uygulama yeniden başlatıldığında silinmez. Temiz bir başlangıç için: `docker compose --profile postgres down -v`

3.7 Testcontainers ile Test

Amaç: Testleri, üretimdekiyle aynı veritabanı motoruna karşı çalıştırmak.

```

@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    // same image as compose.yaml; tell Testcontainers it behaves like the official
    ↪ "postgres" image
    static final DockerImageName IMAGE =
        ↪ DockerImageName.parse("pgvector/pgvector:0.8.6-pg18")
        .asCompatibleSubstituteFor("postgres");

    // static: one container per JVM, shared by every cached Spring test context
    static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(IMAGE);

    @Bean
    @ServiceConnection
    PostgreSQLContainer postgres() {
        return POSTGRES;
    }
}

```

```
}
}
```

Test sınıfları bu konfigürasyonu @Import eder:

Anotasyon	Yüklenen	Transaction
@JdbcTest	DataSource, JdbcClient, Flyway	Her test sonunda rollback
@DataJdbcTest	+ Spring Data JDBC repository'leri	Her test sonunda rollback
@SpringBootTest	Tüm uygulama	Gerçek commit. Veriyi test kendisi temizler

! Uyarı

Container static olduğu için tüm test sınıfları aynı veritabanını paylaşır. Commit eden testler (@SpringBootTest) veriyi @BeforeEach/@AfterEach ile temizlemeli veya yalnızca kendi satırlarına bakmalıdır.

4. Sık Yapılan Hatalar ve En İyi Pratikler

! Uyarı

PostgreSQL'de bir transaction içinde bir SQL hatası olursa, aynı transaction'daki sonraki tüm komutlar reddedilir (current transaction is aborted). Bir testte birden fazla kısıt ihlalini denemek istiyorsanız her birini ayrı bir teste yazın.

- **Yapın:** SQL parametrelerini her zaman :isim veya ? ile verin. Metin birleştirerek SQL kurmayın (SQL injection).
- **Yapmayın:** @Transactional metodu aynı sınıfın içinden çağırmayın. Proxy devreye girmez (self-invocation).
- **Yapın:** İş kurallarını (stok ≥ 0 gibi) hem kodda hem de veritabanında check kısıtı olarak koruyun.
- **Yapmayın:** Checked exception'larda rollback olacağını varsaymayın. Gerekirse rollbackFor kullanın.
- **Yapın:** Yalnızca okuma yapan servis metotlarında @Transactional(readOnly = true) kullanın.
- **Yapmayın:** Testlerde H2 ile "PostgreSQL gibi" davranmayı beklemeyin. Gerçek veritabanını Testcontainers ile kullanın.

5. Özet

- Docker Compose desteği geliştirme veritabanını, Testcontainers ise test veritabanını otomatik başlatır.
- Flyway, şemayı sürümlü SQL dosyalarıyla yönetir. Uygulanan migration değiştirilmez.
- JdbcClient SQL'i sade bir API ile çalıştırır ve sonuçları record'lara eşler.
- Spring Data JDBC, aggregate'leri tek parça olarak saklar. Repository arayüzlerini Spring üretir.
- @Transactional hepsi-ya-hiçbiri garantisi verir. REQUIRES_NEW bağımsız bir transaction açar.
- @TransactionalEventListener(AFTER_COMMIT), yalnızca başarılı commit'lere tepki verir.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Framework — JDBC Core \(JdbcClient\)](#)
- [Spring Framework — Transaction Propagation](#)
- [Spring Framework — Transaction-bound Events](#)
- [Spring Data JDBC](#)
- [Spring Boot — SQL Databases · Database Initialization \(Flyway\)](#)
- [Spring Boot — Testcontainers · Docker Compose Support](#)
- [Flyway Documentation](#)